

COMPTES RENDUS DE TRAVAUX PRATIQUES

Module : PL/SQL et Administration Oracle

Filière : ILISI

Enseignant : Pr. O EL BEGGAR

Environnement : Docker Oracle XE 21c — GitHub Codespaces

7 Travaux Pratiques — Avril 2026

TP	Titre
Manip 1	Créer une base de données Oracle manuellement
Manip 2	Multiplexage et Sauvegarde du fichier de contrôle
Manip 3	Multiplexage des fichiers de journalisation (Redo Logs)
Manip 4	Multiplexage des fichiers de contrôle grâce au PFILE
Chapitre 4	Gérer l'instance Oracle
Chapitre 5	Gérer les structures de stockage — Tablespaces
Chapitre 6	Administrer la sécurité utilisateur
Chapitre 7	Gérer les objets de schéma

Contents

TP 1 — Créer une base de données Oracle manuellement	5
1 Objectif	5
2 Étapes réalisées	5
2.1 Étape 1 — Création de l'arborescence	5
2.2 Étape 2 — Création du fichier INIT.ORA	5
2.3 Étape 3 — Création du SPFILE	5
2.4 Étape 4 — Vérification des composants de la base	6
2.5 Étape 5 — Création d'un tablespace supplémentaire	6
2.6 Étape 6 — Activation du mode ARCHIVELOG	6
3 Interprétation des résultats	7
4 Conclusion	7
TP 2 — Multiplexage et Sauvegarde du fichier de contrôle	8
1 Objectif	8
2 Rappel théorique	8
3 Étapes réalisées	8
3.1 Étape 1 — Vérification de l'état initial	8
3.2 Étape 2 — Modification du paramètre CONTROL_FILES	8
3.3 Étape 3 — Arrêt de la base de données	9
3.4 Étape 4 — Copie physique des fichiers de contrôle	9
3.5 Étape 5 — Redémarrage et vérification	9
3.6 Étape 6 — Sauvegarde du fichier de contrôle en TRACE	9
4 Interprétation des résultats	10
5 Conclusion	10
TP 3 — Multiplexage des fichiers de journalisation	11
1 Objectif	11
2 Rappel théorique	11
3 Étapes réalisées	11
3.1 Étape 1 — État initial des groupes et membres	11
3.2 Étape 2 — Ajout d'un 2 ^{ème} membre à chaque groupe	12
3.3 Étape 3 — Ajout d'un nouveau groupe de Redo Log	12
3.4 Étape 4 — Suppression du groupe 3	12
3.5 Étape 5 — Suppression d'un membre du groupe 1	12
3.6 Étape 6 — Forcer un basculement (Log Switch)	13
4 Interprétation des résultats	13
5 Conclusion	13
TP 4 — Multiplexage des fichiers de contrôle grâce au PFILE	14
1 Objectif	14
2 Rappel théorique	14

3	Étapes réalisées	14
3.1	Étape 1 — Vérification de l'état initial	14
3.2	Étape 2 — Création du PFILE à partir du SPFILE	14
3.3	Étape 3 — Arrêt de l'instance	15
3.4	Étape 4 — Copie physique du nouveau fichier de contrôle	15
3.5	Étape 5 — Modification du PFILE pour ajouter control04	15
3.6	Étape 6 — Démarrage en NOMOUNT et recréation du SPFILE	15
3.7	Étape 7 — Redémarrage complet de l'instance	15
3.8	Étape 8 — Vérification finale du multiplexage	15
4	Interprétation des résultats	16
5	Conclusion	16
TP 5	Gérer l'instance Oracle	18
1	Objectif	18
2	Étapes réalisées	18
2.1	Étape 1 — Afficher et modifier JOB_QUEUE_PROCESSES	18
2.2	Étape 2 — Arrêt et redémarrage de l'instance	18
2.3	Étape 3 — Phases de démarrage dans le fichier d'alertes	18
2.4	Étape 4 — Test de connexion utilisateur HR	19
3	Interprétation des résultats	19
4	Conclusion	19
TP 6	Gérer les structures de stockage	20
1	Objectif	20
2	Étapes réalisées	20
2.1	Étape 1 — Création de l'utilisateur DBA1	20
2.2	Étape 2 — Vérification des tablespaces existants	20
2.3	Étape 3 — Création du tablespace INVENTORY	20
2.4	Étape 4 — Test de la limite d'espace (5 MB)	20
2.5	Étape 5 — Extension du tablespace à 200 MB	21
3	Interprétation des résultats	21
4	Conclusion	21
TP 7	Administrer la sécurité utilisateur	22
1	Objectif	22
2	Étapes réalisées	22
2.1	Étape 1 — Création du profil HRPROFILE	22
2.2	Étape 2 — Création des rôles HRCLERK et HRMANAGER	22
2.3	Étape 3 — Création des utilisateurs RH	22
2.4	Étape 4 — Test des privilèges	23
3	Interprétation des résultats	23
4	Conclusion	24
TP 8	Gérer les objets de schéma	25
1	Objectif	25
2	Étapes réalisées	25
2.1	Étape 1 — Création de la table PRODUCT_MASTER	25
2.2	Étape 2 — Création des tables PRODUCT_ON_HAND et OBSOLETE_PRODUCTS	25
2.3	Étape 3 — Création des index	25
2.4	Étape 4 — Modifications structurelles	26

2.5	Étape 5 — Création de la vue WAREHOUSE_VW	26
2.6	Étape 6 — Vérification des objets créés	26
3	Interprétation des résultats	27
4	Conclusion	27

TP 1 — Créer une base de données Oracle manuellement

1. Objectif

Découvrir les étapes de création d'une base de données Oracle et ses différents composants en la créant manuellement (*from scratch*). Ce TP permet de comprendre l'architecture interne d'Oracle : fichiers de données, fichiers de contrôle, fichiers de journalisation (redo logs), tablespaces, et le mode ARCHIVELOG.

2. Étapes réalisées

2.1 Étape 1 — Création de l'arborescence

Création des répertoires nécessaires au fonctionnement de la base de données :

```
mkdir -p /opt/oracle/admin/mydb/adump
mkdir -p /opt/oracle/admin/mydb/dpdump
mkdir -p /opt/oracle/admin/mydb/pfile
mkdir -p /opt/oracle/oradata/mydb/CONTROLFILE
mkdir -p /opt/oracle/cfgtoollogs/dbca/mydb
mkdir -p /opt/oracle/fast_recovery_area
mkdir -p /opt/oracle/archive
```

Résultat : répertoires adump, dpdump, pfile créés avec succès.

2.2 Étape 2 — Création du fichier INIT.ORA

Le fichier de paramètres INIT.ORA définit la configuration de l'instance Oracle :

Paramètre	Valeur	Rôle
db_name	mydb	Nom de la base
db_block_size	8192	Taille des blocs (8 Ko)
shared_pool_size	536870912	Mémoire pool partagé (512 Mo)
compatible	21.0.0	Version de compatibilité
control_files	control01.ctl, control02.ctl	Fichiers de contrôle
undo_tablespace	UNDOTBS1	Tablespace UNDO
log_archive_dest_1	/opt/oracle/archive	Destination des archives
remote_login_password	EXCLUSIVE	Fichier de mots de passe

2.3 Étape 3 — Création du SPFILE

```
SQL> CREATE SPFILE='/opt/oracle/dbs/spfilemydb.ora'
      FROM PFILE='/opt/oracle/admin/mydb/pfile/init.ora';
```

```
File created.
```

Note : Oracle XE ne permettant qu'une seule instance active, le SPFILE a été créé dans un emplacement dédié à mydb sans remplacer le SPFILE de l'instance XE.

2.4 Étape 4 — Vérification des composants de la base

```
SQL> SELECT instance_name, status FROM v$instance;
INSTANCE_NAME      STATUS
-----
XE                  OPEN

SQL> SELECT name FROM v$datafile;
/opt/oracle/oradata/XE/system01.dbf
/opt/oracle/oradata/XE/sysaux01.dbf
/opt/oracle/oradata/XE/undotbs01.dbf
/opt/oracle/oradata/XE/users01.dbf
... (11 fichiers au total)

SQL> SELECT member FROM v$logfile;
/opt/oracle/oradata/XE/redo01.log
/opt/oracle/oradata/XE/redo02.log
/opt/oracle/oradata/XE/redo03.log

SQL> SHOW PARAMETER control_files;
NAME                TYPE        VALUE
-----
control_files       string      /opt/oracle/oradata/XE/control01.ctl
```

2.5 Étape 5 — Création d'un tablespace supplémentaire

```
SQL> CREATE TABLESPACE USERS2 LOGGING
      DATAFILE '/opt/oracle/oradata/XE/users02.dbf'
      SIZE 5M AUTOEXTEND ON NEXT 1M;
Tablespace created.

SQL> SELECT tablespace_name, status FROM dba_tablespaces;
TABLESPACE_NAME      STATUS
-----
SYSTEM               ONLINE
SYSaux               ONLINE
UNDOTBS1             ONLINE
TEMP                 ONLINE
USERS                ONLINE
USERS2               ONLINE  -- cree avec succes
```

2.6 Étape 6 — Activation du mode ARCHIVELOG

```
SQL> SHUTDOWN IMMEDIATE;
Database closed. Database dismounted. ORACLE instance shut down.

SQL> STARTUP MOUNT;
ORACLE instance started.
Total System Global Area 1610612120 bytes
Database mounted.

SQL> ALTER DATABASE ARCHIVELOG;
Database altered.
```

```
SQL> ALTER DATABASE OPEN;
Database altered.

SQL> SELECT log_mode FROM v$database;
LOG_MODE
-----
ARCHIVELOG
```

3. Interprétation des résultats

► Structure de la base de données

La base Oracle XE contient 11 datafiles répartis entre la base racine (CDB) et les bases pluggables (PDB) : XEPDB1 et PDB\$SEED. Cette architecture multitenant est propre à Oracle 21c. Les fichiers essentiels (SYSTEM, SYSAUX, UNDOTBS1, USERS) sont présents.

► Fichiers de journalisation (Redo Logs)

La base dispose de 3 groupes de redo logs. Ces fichiers enregistrent toutes les modifications apportées à la base. Le processus LGWR écrit en séquence dans ces fichiers de manière circulaire.

► Fichier de contrôle

Il n'existe qu'un seul fichier de contrôle (`control01.ctl`). C'est une situation risquée : si ce fichier est corrompu, la base ne peut plus démarrer. Le Manip 2 corrigera cette situation par le multiplexage.

► Mode ARCHIVELOG

L'activation du mode ARCHIVELOG est cruciale en production. Elle permet de conserver une copie des redo logs avant leur écrasement, rendant possible une récupération complète jusqu'au dernier commit.

► Tablespace USERS2

L'option `AUTOEXTEND ON NEXT 1M` permet une extension automatique du fichier de données, évitant les erreurs ORA-01653 (espace insuffisant).

4. Conclusion

Ce TP nous a permis de comprendre et de réaliser les étapes fondamentales de la création et de la configuration d'une base de données Oracle. Les concepts clés abordés sont : l'arborescence Oracle, les fichiers de paramètres (PFILE/SPFILE), les vues dynamiques de performance (V\$), la gestion des tablespaces, et le mode ARCHIVELOG.

TP 2 — Multiplexage et Sauvegarde du fichier de contrôle

1. Objectif

L'objectif est de sécuriser la base de données Oracle par le multiplexage du fichier de contrôle. Le fichier de contrôle est un fichier critique : sans lui, la base ne peut pas démarrer.

Sans fichier de contrôle, la base ne peut pas démarrer.

Il est recommandé d'avoir au moins 2 fichiers de contrôle sur des disques différents.

2. Rappel théorique

Information	Description
Nom de la base	Identifiant unique de la base de données
Horodatage de création	Date et heure de création de la base
Noms des datafiles	Chemins de tous les fichiers de données
Noms des redo logs	Chemins de tous les fichiers de journalisation
Numéro SCN	System Change Number — numéro de séquence de modification
Informations RMAN	Historique des sauvegardes (Recovery Manager)

3. Étapes réalisées

3.1 Étape 1 — Vérification de l'état initial

```
SQL> SELECT name FROM v$controlfile;
NAME
-----
/opt/oracle/oradata/XE/control01.ctl
1 row selected.

SQL> SHOW PARAMETER control_files;
NAME                TYPE        VALUE
-----
control_files       string      /opt/oracle/oradata/XE/control01.ctl
```

Observation : Un seul fichier de contrôle existe. Sa perte ou corruption rendrait la base inutilisable.

3.2 Étape 2 — Modification du paramètre CONTROL_FILES

```
SQL> ALTER SYSTEM SET control_files=
```



```

'/opt/oracle/oradata/XE/control01.ctl',
'/opt/oracle/oradata/XE/control02.ctl',
'/opt/oracle/fast_recovery_area/control03.ctl'
SCOPE=SPFILE;
System altered.

```

SCOPE=SPFILE : le changement sera effectif au prochain démarrage.

3.3 Étape 3 — Arrêt de la base de données

```

SQL> SHUTDOWN IMMEDIATE;
Database closed. Database dismounted. ORACLE instance shut down.

```

3.4 Étape 4 — Copie physique des fichiers de contrôle

```

$ cp /opt/oracle/oradata/XE/control01.ctl \
    /opt/oracle/oradata/XE/control02.ctl
$ cp /opt/oracle/oradata/XE/control01.ctl \
    /opt/oracle/fast_recovery_area/control03.ctl

$ ls -la /opt/oracle/oradata/XE/control*.ctl
-rw-r----- 1 oracle oinstall 18726912 Apr 9 22:35 control01.ctl
-rw-r----- 1 oracle oinstall 18726912 Apr 9 22:36 control02.ctl
$ ls -la /opt/oracle/fast_recovery_area/control*.ctl
-rw-r----- 1 oracle oinstall 18726912 Apr 9 22:36 control03.ctl

```

3.5 Étape 5 — Redémarrage et vérification

```

SQL> STARTUP;
ORACLE instance started.
Total System Global Area 1610612120 bytes
Fixed Size 9686424 bytes Variable Size 419430400 bytes
Database Buffers 1174405120 bytes Redo Buffers 7090176 bytes
Database mounted. Database opened.

SQL> SELECT name FROM v$controlfile;
NAME
-----
/opt/oracle/oradata/XE/control01.ctl
/opt/oracle/oradata/XE/control02.ctl
/opt/oracle/fast_recovery_area/control03.ctl
3 rows selected.

```

Fichier	Emplacement	Taille	Statut
control01.ctl	/opt/oracle/oradata/XE/	18.7 MB	✓ Actif
control02.ctl	/opt/oracle/oradata/XE/	18.7 MB	✓ Actif
control03.ctl	/opt/oracle/fast_recovery_area/	18.7 MB	✓ Actif

3.6 Étape 6 — Sauvegarde du fichier de contrôle en TRACE

```

SQL> ALTER DATABASE BACKUP CONTROLFILE TO TRACE;
Database altered.
-- Fichier trace genere : XE_ora_3765.trc
-- Emplacement : /opt/oracle/diag/rdbms/xe/XE/trace/

-- Contenu cle du fichier trace (script NORESETLOGS) :
STARTUP NOMOUNT

```

```
CREATE CONTROLFILE REUSE DATABASE "XE" NORESETLOGS ARCHIVELOG
MAXLOGFILES 16 MAXLOGMEMBERS 3 MAXDATAFILES 1024 MAXINSTANCES 8
LOGFILE
GROUP 1 '/opt/oracle/oradata/XE/redo01.log' SIZE 200M,
GROUP 2 '/opt/oracle/oradata/XE/redo02.log' SIZE 200M,
GROUP 3 '/opt/oracle/oradata/XE/redo03.log' SIZE 200M
DATAFILE
'/opt/oracle/oradata/XE/system01.dbf',
'/opt/oracle/oradata/XE/users01.dbf',
... (12 datafiles au total)
CHARACTER SET AL32UTF8;
```

4. Interprétation des résultats

► Importance du multiplexage

Avant le TP, la base ne possédait qu'un seul fichier de contrôle. Après manipulation, elle en possède 3 copies synchronisées automatiquement. Si l'un est corrompu, les deux autres prennent le relais sans interruption de service.

► Synchronisation automatique

Oracle maintient en miroir tous les fichiers listés dans `CONTROL_FILES`. Toute modification de la structure de la base est automatiquement enregistrée dans tous les fichiers simultanément.

► Emplacement stratégique

Les trois fichiers sont sur deux zones différentes : deux dans `oradata` et un dans `fast_recovery_area`. En production, ils devraient être sur des disques physiques différents.

► Fichier TRACE — outil de récupération

Le fichier trace contient un script SQL complet pour recréer le fichier de contrôle *from scratch*. Il contient deux scénarios : `NORESETLOGS` (redo logs disponibles) et `RESETLOGS` (redo logs endommagés).

► Procédure de multiplexage — ordre des opérations

L'ordre est critique : (1) modifier le SPFILE avec `SCOPE=SPFILE`, (2) arrêter la base, (3) copier physiquement les fichiers, (4) redémarrer. Tout déséquilibre entraîne une erreur ORA- au démarrage.

5. Conclusion

Ce TP nous a permis de maîtriser le multiplexage des fichiers de contrôle Oracle et la génération d'une sauvegarde sous forme de script trace. La base dispose maintenant de 3 fichiers de contrôle synchronisés, garantissant une haute disponibilité.

TP 3 — Multiplexage des fichiers de journalisation (Redo Logs)

1. Objectif

Effectuer les opérations d'administration sur les fichiers de redo log : ajouter des membres pour multiplexer les groupes existants, ajouter de nouveaux groupes, et supprimer des groupes ou membres inutiles.

2. Rappel théorique

Statut	Description
CURRENT	Groupe en cours d'écriture par LGWR
ACTIVE	Groupe utilisé pour la récupération d'instance, pas encore archivé
INACTIVE	Groupe archivé, disponible pour réutilisation
UNUSED	Groupe jamais utilisé (nouveau groupe)
INVALID	Membre inaccessible ou nouvellement créé (pas encore utilisé)

Règles importantes :

- La base doit avoir au minimum 2 groupes de redo log.
- Le groupe CURRENT ne peut pas être supprimé.
- En mode ARCHIVELOG, un groupe non encore archivé ne peut pas être supprimé.
- Les fichiers supprimés logiquement doivent être supprimés manuellement sur l'OS.

3. Étapes réalisées

3.1 Étape 1 — État initial des groupes et membres

```
SQL> SELECT group#, members, status FROM v$log;
GROUP#    MEMBERS STATUS
-----
        1         1 INACTIVE
        2         1  CURRENT
        3         1 INACTIVE

SQL> SELECT group#, member FROM v$logfile ORDER BY group#;
GROUP#  MEMBER
-----
        1 /opt/oracle/oradata/XE/redo01.log
        2 /opt/oracle/oradata/XE/redo02.log
        3 /opt/oracle/oradata/XE/redo03.log
```

Observation : 3 groupes avec 1 seul membre chacun. Aucun multiplexage. Si un fichier redo log est perdu, Oracle s'arrête.

3.2 Étape 2 — Ajout d'un 2^{ème} membre à chaque groupe

```
SQL> ALTER DATABASE ADD LOGFILE MEMBER
      '/opt/oracle/oradata/XE/redo01b.log' TO GROUP 1;
Database altered.
SQL> ALTER DATABASE ADD LOGFILE MEMBER
      '/opt/oracle/oradata/XE/redo02b.log' TO GROUP 2;
Database altered.
SQL> ALTER DATABASE ADD LOGFILE MEMBER
      '/opt/oracle/oradata/XE/redo03b.log' TO GROUP 3;
Database altered.

SQL> SELECT group#, member, status FROM v$logfile ORDER BY group#;
GROUP#  MEMBER                                     STATUS
-----  -
1      /opt/oracle/oradata/XE/redo01.log
1      /opt/oracle/oradata/XE/redo01b.log             INVALID
2      /opt/oracle/oradata/XE/redo02.log
2      /opt/oracle/oradata/XE/redo02b.log             INVALID
3      /opt/oracle/oradata/XE/redo03.log
3      /opt/oracle/oradata/XE/redo03b.log             INVALID
6 rows selected.
```

Note : Statut INVALID car les fichiers viennent d'être créés. Ils deviendront valides lors du prochain cycle d'écriture.

3.3 Étape 3 — Ajout d'un nouveau groupe de Redo Log

```
SQL> ALTER DATABASE ADD LOGFILE GROUP 4
      ('/opt/oracle/oradata/XE/redo04a.log',
       '/opt/oracle/oradata/XE/redo04b.log') SIZE 200M;
Database altered.

SQL> SELECT group#, members, status FROM v$log;
GROUP#  MEMBERS STATUS
-----  -
1      2 INACTIVE
2      2 CURRENT
3      2 INACTIVE
4      2 UNUSED
```

3.4 Étape 4 — Suppression du groupe 3

```
SQL> ALTER DATABASE DROP LOGFILE GROUP 3;
Database altered.

$ rm /opt/oracle/oradata/XE/redo03.log
$ rm /opt/oracle/oradata/XE/redo03b.log
$ ls /opt/oracle/oradata/XE/redo*.log
redo01.log redo01b.log redo02.log redo02b.log redo04a.log redo04b.log
```

3.5 Étape 5 — Suppression d'un membre du groupe 1

```
SQL> ALTER DATABASE DROP LOGFILE MEMBER
      '/opt/oracle/oradata/XE/redo01b.log';
Database altered.
$ rm /opt/oracle/oradata/XE/redo01b.log
```

3.6 Étape 6 — Forcer un basculement (Log Switch)

```
SQL> ALTER SYSTEM SWITCH LOGFILE;
System altered.

SQL> SELECT group#, members, status FROM v$log;
GROUP#    MEMBERS STATUS
-----
1          1 INACTIVE
2          2 ACTIVE
4          2 CURRENT
```

Récapitulatif de l'état final des redo logs :

Groupe	Membres	Fichiers	Statut
1	1	redo01.log	INACTIVE
2	2	redo02.log, redo02b.log	ACTIVE
4	2	redo04a.log, redo04b.log	CURRENT

4. Interprétation des résultats

► Multiplexage des membres

En ajoutant un 2^{ème} membre, LGWR écrit simultanément dans les deux. Si un membre est inaccessible, Oracle continue avec l'autre sans interruption. C'est le principe du mirroring au niveau des redo logs.

► Statut INVALID des nouveaux membres

Ce statut indique que le fichier existe physiquement mais n'a pas encore été écrit par LGWR. Il disparaît automatiquement lors du prochain cycle d'écriture sur ce groupe.

► Ajout d'un nouveau groupe

Le groupe 4 créé avec 2 membres a le statut UNUSED. Après le log switch, il est passé en CURRENT, confirmant son intégration dans la rotation circulaire.

► Suppression du groupe 3

Opération en deux temps : suppression logique dans Oracle puis suppression physique sur le disque. Oracle ne supprime jamais automatiquement les fichiers OS pour des raisons de sécurité.

► Log Switch — basculement forcé

ALTER SYSTEM SWITCH LOGFILE force Oracle à archiver le groupe CURRENT et passer au suivant. Utile avant une sauvegarde pour s'assurer que toutes les modifications sont archivées.

5. Conclusion

Ce TP nous a permis de maîtriser la gestion complète des fichiers de redo log Oracle. La combinaison du multiplexage des fichiers de contrôle (TP 2) et des redo logs (TP 3) constitue la base d'une architecture Oracle sécurisée.

TP 4 — Multiplexage des fichiers de contrôle grâce au PFILE

1. Objectif

Créer un fichier de paramètres PFILE à partir du SPFILE existant, le modifier pour ajouter un quatrième fichier de contrôle, puis régénérer le SPFILE à partir du PFILE modifié. Cette approche permet de multiplexer le fichier de contrôle via le PFILE, renforçant la sécurité de la base.

2. Rappel théorique

Critère	PFILE (init.ora)	SPFILE (spfile.ora)
Type	Fichier texte éditable	Fichier binaire géré par Oracle
Modification	Éditeur de texte / sed	ALTER SYSTEM SET ... SCOPE=SPFILE
Emplacement	\$ORACLE_HOME/dbs/ ou custom	\$ORACLE_HOME/dbs/ (défaut)
Utilisation	STARTUP PFILE='chemin'	Démarrage automatique
Avantage	Facile à éditer et auditer	Modifié dynamiquement

3. Étapes réalisées

3.1 Étape 1 — Vérification de l'état initial

```
SQL> SELECT name FROM v$controlfile;
NAME
-----
/opt/oracle/oradata/XE/control01.ctl
/opt/oracle/oradata/XE/control02.ctl
/opt/oracle/fast_recovery_area/control03.ctl
3 rows selected.
```

Observation : 3 fichiers de contrôle issus du TP 2. Objectif : ajouter un 4^{ème} via le PFILE.

3.2 Étape 2 — Création du PFILE à partir du SPFILE

```
SQL> CREATE PFILE='/opt/oracle/admin/XE/pfile/init_manip4.ora'
FROM SPFILE;
File created.

$ cat /opt/oracle/admin/XE/pfile/init_manip4.ora
XE.__data_transfer_cache_size=0
XE.__db_cache_size=1090519040
...
*.control_files='/opt/oracle/oradata/XE/control01.ctl',
```

```
        '/opt/oracle/oradata/XE/control02.ctl',
        '/opt/oracle/fast_recovery_area/control03.ctl'
*.db_block_size=8192
*.db_name='XE'
```

3.3 Étape 3 — Arrêt de l'instance

```
SQL> SHUTDOWN IMMEDIATE;
Database closed. Database dismounted. ORACLE instance shut down.
```

3.4 Étape 4 — Copie physique du nouveau fichier de contrôle

```
$ cp /opt/oracle/oradata/XE/control01.ctl \
    /opt/oracle/oradata/XE/control04.ctl

$ ls -lh /opt/oracle/oradata/XE/control*.ctl
-rw-r----- 1 oracle oinstall 18M Apr  9 23:20 control01.ctl
-rw-r----- 1 oracle oinstall 18M Apr  9 23:20 control02.ctl
-rw-r----- 1 oracle oinstall 18M Apr  9 23:22 control04.ctl
```

3.5 Étape 5 — Modification du PFILE pour ajouter control04

```
$ sed -i "s|*.control_files=.*|*.control_files=
'/opt/oracle/oradata/XE/control01.ctl',
'/opt/oracle/oradata/XE/control02.ctl',
'/opt/oracle/fast_recovery_area/control03.ctl',
'/opt/oracle/oradata/XE/control04.ctl'|" \
/opt/oracle/admin/XE/pfile/init_manip4.ora

$ grep control_files /opt/oracle/admin/XE/pfile/init_manip4.ora
*.control_files='/opt/oracle/oradata/XE/control01.ctl',
'/opt/oracle/oradata/XE/control02.ctl',
'/opt/oracle/fast_recovery_area/control03.ctl',
'/opt/oracle/oradata/XE/control04.ctl'
```

3.6 Étape 6 — Démarrage en NOMOUNT et recréation du SPFILE

```
SQL> STARTUP NOMOUNT PFILE='/opt/oracle/admin/XE/pfile/init_manip4.ora';
ORACLE instance started.
Total System Global Area 1610612120 bytes
Fixed Size 9686424 bytes Variable Size 419430400 bytes
Database Buffers 1174405120 bytes Redo Buffers 7090176 bytes

SQL> CREATE SPFILE FROM PFILE='/opt/oracle/admin/XE/pfile/init_manip4.ora';
File created.
```

3.7 Étape 7 — Redémarrage complet de l'instance

```
SQL> SHUTDOWN IMMEDIATE;
ORACLE instance shut down.

SQL> STARTUP;
ORACLE instance started.
...
Database mounted.

SQL> ALTER DATABASE OPEN;
Database altered.
```

3.8 Étape 8 — Vérification finale du multiplexage

```

SQL> SELECT name FROM v$controlfile;
NAME
-----
/opt/oracle/oradata/XE/control01.ctl
/opt/oracle/oradata/XE/control02.ctl
/opt/oracle/fast_recovery_area/control03.ctl
/opt/oracle/oradata/XE/control04.ctl
4 rows selected.

SQL> SHOW PARAMETER control_files;
NAME                TYPE        VALUE
-----
control_files       string      /opt/oracle/oradata/XE/control01.ctl,
                  /opt/oracle/oradata/XE/control02.ctl,
                  /opt/oracle/fast_recovery_area/control03.ctl,
                  /opt/oracle/oradata/XE/control04.ctl

```

Fichier	Emplacement	Taille	Statut
control01.ctl	/opt/oracle/oradata/XE/	18 MB	✓ Actif
control02.ctl	/opt/oracle/oradata/XE/	18 MB	✓ Actif
control03.ctl	/opt/oracle/fast_recovery_area/	18 MB	✓ Actif
control04.ctl	/opt/oracle/oradata/XE/	18 MB	✓ Actif

4. Interprétation des résultats

► Rôle du PFILE comme intermédiaire

Contrairement au TP 2 (ALTER SYSTEM SCOPE=SPFILE), ce TP utilise le PFILE comme intermédiaire. Cette approche est plus transparente : on valide la configuration dans un fichier texte avant de l'appliquer.

► Ordre critique des opérations

(1) Créer le PFILE, (2) arrêter la base, (3) copier physiquement le fichier, (4) modifier le PFILE, (5) démarrer en NOMOUNT avec PFILE, (6) recréer le SPFILE, (7) redémarrer.

► Mode NOMOUNT pour la création du SPFILE

Oracle doit être actif pour exécuter CREATE SPFILE, mais la base ne doit pas être montée pour éviter les conflits avec l'ancien SPFILE.

► Synchronisation automatique des 4 fichiers

Oracle maintient les 4 fichiers en miroir de façon transparente. Toute modification structurelle est automatiquement propagée par le processus CKPT.

► Comparaison TP 2 vs TP 4

TP 2 : rapide via ALTER SYSTEM. TP 4 : plus pédagogique, auditable et reproductible via PFILE. Les deux approches sont valides en production selon le contexte.

5. Conclusion

La base dispose désormais de 4 fichiers de contrôle synchronisés.

TP	Titre	Compétences acquises
TP 1	Création manuelle de la base	Architecture Oracle, SPFILE, ARCHIVELOG
TP 2	Multiplexage fichier de contrôle (SP-FILE)	ALTER SYSTEM SCOPE=SPFILE, copie physique
TP 3	Multiplexage des redo logs	ADD/DROP LOGFILE, SWITCH LOG-FILE
TP 4	Multiplexage fichier de contrôle (PFILE)	CREATE PFILE FROM SPFILE, sed, NOMOUNT

TP 5 — Gérer l'instance Oracle

1. Objectif

Vérifier le démarrage et l'arrêt de la base de données Oracle, modifier les paramètres d'initialisation, et consulter les phases de démarrage dans le fichier d'alertes. Ce TP s'effectue dans un environnement Docker Oracle XE 21c sur GitHub Codespaces en remplacement d'Enterprise Manager 10g.

2. Étapes réalisées

2.1 Étape 1 — Afficher et modifier JOB_QUEUE_PROCESSES

```
SQL> SHOW PARAMETER job_queue_processes;
NAME                                TYPE          VALUE
-----
job_queue_processes                 integer       40

SQL> ALTER SYSTEM SET JOB_QUEUE_PROCESSES = 15;
System altered.

SQL> SHOW PARAMETER job_queue_processes;
NAME                                TYPE          VALUE
-----
job_queue_processes                 integer       15
```

Q : Instruction SQL exécutée : ALTER SYSTEM SET JOB_QUEUE_PROCESSES = 15;

Q : Colonne Dynamic : Un paramètre Dynamic peut être modifié sans redémarrer l'instance (effet immédiat). Un paramètre statique nécessite SCOPE=SPFILE et un redémarrage.

2.2 Étape 2 — Arrêt et redémarrage de l'instance

```
SQL> SHUTDOWN IMMEDIATE;
Database closed. Database dismounted. ORACLE instance shut down.

SQL> STARTUP;
ORACLE instance started.
Total System Global Area 1610612120 bytes
Fixed Size 9686424 bytes Variable Size 419430400 bytes
Database Buffers 1174405120 bytes Redo Buffers 7090176 bytes
Database mounted. Database opened.
```

Q : Instruction de démarrage : STARTUP;

2.3 Étape 3 — Phases de démarrage dans le fichier d'alertes

```
SQL> SELECT value FROM v$diag_info WHERE name = 'Diag Trace';
VALUE
-----
/opt/oracle/diag/rdbms/xe/XE/trace
```

```
$ tail -50 /opt/oracle/diag/rdbms/xe/XE/trace/alert_XE.log
...ORACLE instance started...
...Database mounted...
PDB$SEED(2):Pluggable database PDB$SEED opening in read only
XEPDB1(3):Pluggable database XEPDB1 opening in read write
Completed: ALTER DATABASE OPEN
```

Q : Phases de démarrage :

- (1) **NOMOUNT** — instance démarrée, SGA allouée, processus background lancés
- (2) **MOUNT** — fichier de contrôle lu, base montée
- (3) **OPEN** — datafiles et redo logs ouverts, base accessible
- (4) **PDBs ouvertes** — PDB\$SEED (read only) et XEPDB1 (read write) ouvertes automatiquement

2.4 Étape 4 — Test de connexion utilisateur HR

HR n'existant pas dans Oracle XE 21c, création dans la PDB XEPDB1 :

```
SQL> ALTER SESSION SET CONTAINER = XEPDB1;
Session altered.
SQL> CREATE USER hr IDENTIFIED BY hr;
User created.
SQL> GRANT CREATE SESSION TO hr;
Grant succeeded.
SQL> CONNECT hr/hr@XEPDB1;
Connected.
```

3. Interprétation des résultats

► Paramètre dynamique vs statique

JOB_QUEUE_PROCESSES est dynamique : modifiable avec **ALTER SYSTEM** sans redémarrage. Les paramètres statiques nécessitent **SCOPE=SPFILE** et un redémarrage.

► Phases de démarrage Oracle 21c

NOMOUNT → MOUNT → OPEN → PDBs ouvertes. Oracle 21c CDB ouvre automatiquement les PDBs configurées au démarrage.

4. Conclusion

Ce TP a permis de maîtriser la gestion de l'instance Oracle : modification des paramètres d'initialisation, arrêt/démarrage, et analyse du fichier d'alertes. L'adaptation à Oracle XE 21c (CDB/PDB) a nécessité la création des utilisateurs dans la PDB XEPDB1.

TP 6 — Gérer les structures de stockage — Tablespaces

1. Objectif

Créer et gérer un tablespace Oracle localement géré nommé INVENTORY, tester les limites d'espace et les étendre dynamiquement. Ce TP remplace les manipulations Enterprise Manager 10g par des commandes SQL directes.

2. Étapes réalisées

2.1 Étape 1 — Création de l'utilisateur DBA1

```
SQL> CREATE USER dba1 IDENTIFIED BY oracle;
User created.
SQL> GRANT DBA TO dba1;
Grant succeeded.
```

2.2 Étape 2 — Vérification des tablespaces existants

```
SQL> SELECT tablespace_name, status, contents, extent_management
FROM dba_tablespaces;
TABLESPACE_NAME  STATUS  CONTENTS  EXTENT_MAN
-----
SYSTEM           ONLINE  PERMANENT LOCAL
SYSAUX           ONLINE  PERMANENT LOCAL
UNDOTBS1         ONLINE  UNDO      LOCAL
TEMP             ONLINE  TEMPORARY LOCAL
USERS            ONLINE  PERMANENT LOCAL
```

Note : Le tablespace EXAMPLE n'existe pas dans Oracle XE 21c (spécifique à Oracle 10g).

2.3 Étape 3 — Création du tablespace INVENTORY

```
SQL> CREATE TABLESPACE INVENTORY
DATAFILE '/opt/oracle/oradata/XE/inventory01.dbf'
SIZE 5M EXTENT MANAGEMENT LOCAL
AUTOALLOCATE SEGMENT SPACE MANAGEMENT AUTO LOGGING;
Tablespace created.

SQL> SELECT tablespace_name, status, extent_management,
segment_space_management
FROM dba_tablespaces WHERE tablespace_name = 'INVENTORY';
TABLESPACE_NAME  STATUS  EXTENT_MAN  SEGMENT
-----
INVENTORY        ONLINE  LOCAL      AUTO
```

2.4 Étape 4 — Test de la limite d'espace (5 MB)

```
SQL> CREATE USER inventory IDENTIFIED BY oracle
DEFAULT TABLESPACE INVENTORY QUOTA UNLIMITED ON INVENTORY;
```

```
SQL> GRANT CREATE SESSION, CREATE TABLE TO inventory;
SQL> CREATE TABLE inventory.test_table (id NUMBER, data VARCHAR2(4000))
      TABLESPACE INVENTORY;
Table created.

SQL> BEGIN
      FOR i IN 1..10000 LOOP
        INSERT INTO inventory.test_table VALUES (i, RPAD('X',4000,'X'));
      END LOOP; COMMIT;
    END; /

ERROR at line 1:
ORA-01653: unable to extend table INVENTORY.TEST_TABLE
          by 128 in tablespace INVENTORY
```

Q : Erreur obtenue : ORA-01653 — espace insuffisant dans le tablespace INVENTORY (5 MB).

2.5 Étape 5 — Extension du tablespace à 200 MB

```
SQL> ALTER DATABASE DATAFILE '/opt/oracle/oradata/XE/inventory01.dbf'
      RESIZE 200M;
Database altered.

SQL> BEGIN
      FOR i IN 1..10000 LOOP
        INSERT INTO inventory.test_table VALUES (i, RPAD('X',4000,'X'));
      END LOOP; COMMIT;
    END; /
PL/SQL procedure successfully completed.

SQL> DROP TABLE inventory.test_table PURGE;
Table dropped.
```

Q : Instruction ALTER : ALTER DATABASE DATAFILE '...' RESIZE 200M;

3. Interprétation des résultats

► Tablespace localement géré (LMT)

EXTENT MANAGEMENT LOCAL stocke la carte des extents dans le tablespace lui-même (bitmap). Plus performant que les tablespaces gérés par dictionnaire (DMT) car pas de contention sur SYSTEM.

► ORA-01653 — Gestion de l'espace

L'erreur signale que le tablespace ne peut plus allouer de nouveaux extents. La solution : RESIZE du datafile, AUTOEXTEND ON, ou ajout d'un nouveau datafile.

4. Conclusion

Ce TP a permis de créer et gérer un tablespace Oracle localement géré, de tester les limites d'espace et de les corriger dynamiquement. Le tablespace INVENTORY sera réutilisé dans les TPs suivants.

TP 7 — Administrer la sécurité utilisateur

1. Objectif

Créer des profils, des rôles et des utilisateurs Oracle pour le département des Ressources Humaines. Mettre en place une politique de déconnexion automatique après 15 minutes d'inactivité et tester les privilèges des différents utilisateurs.

2. Étapes réalisées

2.1 Étape 1 — Création du profil HRPROFILE

```
SQL> CREATE PROFILE hrprofile LIMIT
      IDLE_TIME 15 SESSIONS_PER_USER UNLIMITED CONNECT_TIME UNLIMITED;
Profile created.

SQL> ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;
System altered.
```

2.2 Étape 2 — Création des rôles HRCLERK et HRMANAGER

```
SQL> CREATE ROLE hrclerk;
Role created.
SQL> GRANT SELECT, UPDATE ON hr.employees TO hrclerk;
Grant succeeded.

SQL> CREATE ROLE hrmanager;
Role created.
SQL> GRANT DELETE, INSERT ON hr.employees TO hrmanager;
Grant succeeded.
SQL> GRANT hrclerk TO hrmanager;
Grant succeeded.
SQL> GRANT SELECT, UPDATE, INSERT, DELETE ON hr.employees TO hrmanager;
Grant succeeded.
```

2.3 Étape 3 — Création des utilisateurs RH

```
-- David Hamby (opérateur --- role HRCLERK)
SQL> CREATE USER dhamby IDENTIFIED BY newuser PROFILE hrprofile
      DEFAULT TABLESPACE INVENTORY TEMPORARY TABLESPACE TEMP
      PASSWORD EXPIRE ACCOUNT UNLOCK;
User created.
SQL> GRANT CONNECT TO dhamby; Grant succeeded.
SQL> GRANT hrclerk TO dhamby; Grant succeeded.

-- Rachel Pandya (opératrice --- role HRCLERK)
SQL> CREATE USER rpandya IDENTIFIED BY newuser PROFILE hrprofile
      DEFAULT TABLESPACE INVENTORY TEMPORARY TABLESPACE TEMP
      PASSWORD EXPIRE ACCOUNT UNLOCK;
User created.
SQL> GRANT CONNECT TO rpandya; Grant succeeded.
SQL> GRANT hrclerk TO rpandya; Grant succeeded.
```

```
-- Jenny Goodman (manager --- role HRMANAGER)
SQL> CREATE USER jgoodman IDENTIFIED BY newuser PROFILE hrprofile
      DEFAULT TABLESPACE INVENTORY TEMPORARY TABLESPACE TEMP
      PASSWORD EXPIRE ACCOUNT UNLOCK;
User created.
SQL> GRANT CONNECT TO jgoodman; Grant succeeded.
SQL> GRANT hrmanager TO jgoodman; Grant succeeded.
```

2.4 Étape 4 — Test des privilèges

```
-- Test DHAMBY : SELECT OK, DELETE interdit
SQL> CONNECT dhamby/oracle@XEPDB1; Connected.
SQL> SELECT employee_id, first_name FROM hr.employees WHERE employee_id=50;
EMPLOYEE_ID  FIRST_NAME
-----
50  First50      -- OK

SQL> DELETE FROM hr.employees WHERE employee_id = 50;
ERROR: ORA-01031: insufficient privileges -- Correct !

-- Test JGOODMAN : DELETE autorise
SQL> CONNECT jgoodman/oracle@XEPDB1; Connected.
SQL> DELETE FROM hr.employees WHERE employee_id = 50;
1 row deleted.
SQL> ROLLBACK; Rollback complete.
SQL> SELECT COUNT(*) FROM hr.employees;
COUNT(*)
-----
107
```

Utilisateur	Rôle	SELECT	UPDATE	INSERT	DELETE
DHAMBY	HRCLERK	✓	✓	✗	✗
RPANDYA	HRCLERK	✓	✓	✗	✗
JGOODMAN	HRMANAGER	✓	✓	✓	✓

3. Interprétation des résultats

► Q1 — Où la ligne supprimée est-elle stockée ?

Dans le tablespace UNDO (UNDOTBS1). Oracle conserve l'image avant modification dans les segments d'annulation jusqu'au COMMIT, permettant ainsi le ROLLBACK.

► Q2 — Tablespaces des nouveaux utilisateurs ?

Le tablespace par défaut de la base (DATABASE DEFAULT TABLESPACE) s'applique lorsqu'aucun tablespace n'est spécifié à la création de l'utilisateur.

► Q3 — Connexion sans CREATE SESSION ?

Le rôle CONNECT inclut le privilège CREATE SESSION. En accordant CONNECT, les utilisateurs obtiennent implicitement le droit de se connecter.

► Profil HRPROFILE — déconnexion automatique

IDLE_TIME = 15 provoque la déconnexion après 15 min d'inactivité. RESOURCE_LIMIT doit être TRUE. Vérification :

```
ERROR at line 1:
```

```
ORA-02396: exceeded maximum idle time, please connect again
```

4. Conclusion

Ce TP a permis de mettre en place une politique de sécurité complète : profil avec timeout, rôles hiérarchiques (HRCLERK inclus dans HRMANAGER), et utilisateurs avec privilèges adaptés.

TP 8 — Gérer les objets de schéma

1. Objectif

Créer les objets de schéma pour l'application de gestion des stocks : tables avec contraintes, index (B-tree et Bitmap), vue, et modifications structurelles (ajout/suppression de colonnes et tables).

2. Étapes réalisées

2.1 Étape 1 — Création de la table PRODUCT_MASTER

```
SQL> CREATE TABLE inventory.product_master (  
  product_id      NUMBER(7)      CONSTRAINT pk_inv PRIMARY KEY,  
  product_name    VARCHAR2(50) NOT NULL,  
  code            VARCHAR2(10) NOT NULL,  
  reorder_threshold NUMBER(5)      CONSTRAINT chk_reorder  
                                     CHECK (reorder_threshold > 0),  
  cost            NUMBER(5,2),  
  price           NUMBER(5,2)  
) TABLESPACE INVENTORY;  
Table created.
```

2.2 Étape 2 — Création des tables PRODUCT_ON_HAND et OBSOLETE_PRODUCTS

```
SQL> CREATE TABLE inventory.product_on_hand (  
  product_id      NUMBER(7) CONSTRAINT fk_prod  
                                     REFERENCES inventory.product_master(product_id),  
  quantity        NUMBER(5),  
  warehouse_city  VARCHAR2(30),  
  last_update     DATE) TABLESPACE INVENTORY;  
Table created.  
  
SQL> CREATE TABLE inventory.obsolete_products (  
  product_id      NUMBER(7)      PRIMARY KEY,  
  product_name    VARCHAR2(50) NOT NULL,  
  code            VARCHAR2(20) NOT NULL,  
  cost            NUMBER(5,2), price NUMBER(5,2)) TABLESPACE INVENTORY;  
Table created.
```

2.3 Étape 3 — Création des index

```
-- Index Bitmap sur OBSOLETE_PRODUCTS.CODE (faible cardinalite)  
SQL> CREATE BITMAP INDEX inventory.obs_code  
  ON inventory.obsolete_products(code) TABLESPACE INVENTORY;  
Index created.  
  
-- Index B-tree combine sur PRODUCT_MASTER  
SQL> CREATE INDEX inventory.prod_name_code  
  ON inventory.product_master(product_name, code)  
  TABLESPACE INVENTORY;  
Index created.
```

```
-- Index B-tree combine sur PRODUCT_ON_HAND
SQL> CREATE INDEX inventory.poh_prod_id_qty
      ON inventory.product_on_hand(product_id, quantity)
      TABLESPACE INVENTORY;
Index created.
```

Q : Type d'index pour OBSOLETE_PRODUCTS.CODE : BITMAP — colonne à faible cardinalité. B-tree est préférable pour les colonnes à haute cardinalité.

2.4 Étape 4 — Modifications structurelles

```
-- Ajout de colonnes a PRODUCT_MASTER
SQL> ALTER TABLE inventory.product_master
      ADD (primary_source VARCHAR2(50), secondary_source VARCHAR2(50));
Table altered.

-- Suppression de OBSOLETE_PRODUCTS + ajout colonne OBSOLETE
SQL> DROP TABLE inventory.obsolete_products CASCADE CONSTRAINTS;
Table dropped.
SQL> ALTER TABLE inventory.product_master ADD (obsoleted DATE);
Table altered.
```

Q : Clause de DROP TABLE : CASCADE CONSTRAINTS — supprime les FK référençant la table. Sans elle : ORA-02449.

2.5 Étape 5 — Création de la vue WAREHOUSE_VW

```
SQL> CREATE VIEW inventory.warehouse_vw AS
      SELECT pm.product_name, poh.quantity, poh.warehouse_city
      FROM inventory.product_master pm
      JOIN inventory.product_on_hand poh
        ON pm.product_id = poh.product_id;
View created.
```

2.6 Étape 6 — Vérification des objets créés

```
SQL> SELECT index_name, table_name, index_type FROM dba_indexes
      WHERE owner = 'INVENTORY';
INDEX_NAME          TABLE_NAME          INDEX_TYPE
-----
PK_INV              PRODUCT_MASTER       NORMAL
PROD_NAME_CODE      PRODUCT_MASTER       NORMAL
POH_PROD_ID_QTY     PRODUCT_ON_HAND      NORMAL

SQL> SELECT constraint_name, constraint_type, table_name
      FROM dba_constraints
      WHERE owner='INVENTORY' AND constraint_name NOT LIKE 'BIN$%';
CONSTRAINT_NAME     C  TABLE_NAME
-----
PK_INV              P  PRODUCT_MASTER
CHK_REORDER         C  PRODUCT_MASTER
SYS_C008221         C  PRODUCT_MASTER
SYS_C008222         C  PRODUCT_MASTER
FK_PROD             R  PRODUCT_ON_HAND
```

Objet	Type	Détail
PRODUCT_MASTER	Table	7 colonnes + PRIMARY KEY, CHECK, NOT NULL
PRODUCT_ON_HAND	Table	4 colonnes + FK vers PRODUCT_MASTER
PK_INV	Index B-tree	Clé primaire de PRODUCT_MASTER
PROD_NAME_CODE	Index B-tree	Combiné PRODUCT_NAME + CODE
POH_PROD_ID_QTY	Index B-tree	Combiné PRODUCT_ID + QUANTITY
WAREHOUSE_VW	Vue	Jointure PRODUCT_MASTER + PRODUCT_ON_HAND

3. Interprétation des résultats

► Choix du type d'index

Un index Bitmap est adapté aux colonnes à faible cardinalité (peu de valeurs distinctes). Pour les colonnes à haute cardinalité (IDs, noms), l'index B-tree est plus efficace — recherche en $O(\log n)$.

► CASCADE CONSTRAINTS dans DROP TABLE

Supprime automatiquement toutes les contraintes FK qui référencent la table. Sans cette clause, Oracle renvoie ORA-02449 si des FK existent.

► Vue WAREHOUSE_VW

La vue joint PRODUCT_MASTER et PRODUCT_ON_HAND pour exposer une vision simplifiée des stocks par entrepôt, simplifiant les requêtes et contrôlant l'accès aux données.

► Vue dictionnaire pour les contraintes INVENTORY

USER_CONSTRAINTS pour les objets de l'utilisateur courant ; DBA_CONSTRAINTS pour tous les utilisateurs (nécessite DBA). L'utilisateur INVENTORY possède 5 contraintes actives.

► Nombre d'index de l'utilisateur INVENTORY

L'utilisateur INVENTORY possède **3 index** après le TP (OBS_CODE supprimé avec la table OBSOLETE_PRODUCTS).

4. Conclusion

Ce TP a permis de créer une structure de base de données complète pour l'application de gestion des stocks. Les 4 chapitres couvrent l'ensemble des compétences d'administration Oracle.

TP	Titre	Compétences acquises
TP 5	Gérer l'instance Oracle	ALTER SYSTEM, STARTUP/SHUTDOWN, alertes
TP 6	Gérer les Tablespaces	CREATE TABLESPACE, RESIZE, ORA-01653
TP 7	Sécurité utilisateur	Profils, rôles, utilisateurs, privilèges
TP 8	Objets de schéma	Tables, index, vues, contraintes, ALTER/DROP